

Sentinel

A Unified Runtime Safety Instrumentation Layer for Autonomous LLM Agents

Technical Design Document

v0.1 — Draft

This document specifies the problem, design, and delivery plan for a drop-in software development kit that instruments autonomous LLM agents against the specific class of failure modes currently documented in frontier AI safety research — reasoning unfaithfulness, tool-provenance fabrication, evaluation-aware sandbagging, memory corruption, and specification gaming — and gates consequential agent actions behind a configurable human-oversight checkpoint.

Contents

1	Problem Statement	2
2	Landscape and Prior Art	2
3	Solution Overview	3
4	System Architecture	3
5	Safety Evaluation Modules	4
5.1	Reasoning Faithfulness	4
5.2	Tool-Use Grounding and Provenance	5
5.3	Evaluation-Awareness and Sandbagging	5
5.4	Memory and Context Integrity	5
5.5	Specification Gaming and Objective Drift	6
5.6	Human Oversight and Escalation Gating	6
6	Delivery Plan	6
7	Evaluation Methodology	7
8	Risks and Open Questions	7
9	Conclusion	7

1 Problem Statement

Autonomous agents built on large language models are being deployed into consequential workflows at a pace that has outrun the tooling available to supervise them. An agent that reads documents, calls external tools, updates its own memory, and takes actions on a user's behalf is no longer a text generator that can be spot-checked by reading its output; it is a system with an internal decision process, a persistent state, and an increasingly indirect relationship between what it says and what it does. The organizations deploying these agents overwhelmingly rely on generic application logging, ad hoc prompt inspection, or simple output-quality evaluation, none of which was designed to detect the failure modes that matter most once an agent is autonomous rather than merely generative. This gap is not hypothetical. Over the past two years, a growing body of controlled research from frontier AI labs and independent safety organizations has documented, reproducibly, that language-model agents will produce reasoning traces that do not reflect the actual basis for their decisions, will claim to have retrieved or verified information they never actually retrieved, will behave differently when they infer they are being evaluated than when they infer they are not, will allow their own working memory to drift or be corrupted by a single bad update that then propagates as established fact, and will satisfy the literal wording of an instruction while defeating its evident intent. Each of these failure modes has been demonstrated individually in isolated academic or lab settings. None of them is currently addressed by a single, deployable, engineering-grade instrumentation layer that a team can install into an existing agent codebase without adopting an entirely new orchestration framework or waiting on a research collaboration. The problem this document addresses is the absence of that layer: a practical, opinionated, installable translation of the current AI safety literature into a monitoring and gating system that a team shipping an agent today can actually run in production.

The cost of this gap is not abstract. An agent that fabricates a tool result and is trusted on that basis can cause a financial, legal, or safety-relevant error that is discovered only after the fact, at which point the organization has no artifact — no trace, no flag, no audit record — that would have let a human catch it before it mattered. An agent that behaves well under evaluation and differently in production defeats the entire premise of pre-deployment testing. An agent whose memory silently absorbs and then repeats a fabricated or manipulated claim turns a single bad inference into a persistent, load-bearing false belief that no amount of prompt engineering at the point of failure will fix, because the failure already happened upstream, in memory, out of view. The teams building and deploying these systems are not unaware of this risk in the abstract; what they lack is a concrete, low-integration-cost way to see it happening.

2 Landscape and Prior Art

The closest existing category of tooling is LLM observability, represented by products such as LangSmith, Langfuse, Arize Phoenix, Braintrust, Galileo, and Weights and Biases Weave. These tools solve a real and adjacent problem well: they capture traces, latency, token cost, and output quality across chains and agent runs, and several of them offer generic hallucination scoring or output-similarity evaluation against a reference. They are, however, built around the observability paradigm of conventional software — logs, traces, spans, dashboards — extended with a layer of LLM-specific quality scoring. None of the major products in this category implement the specific, named failure modes that current safety research has isolated as distinct phenomena with distinct detection methodologies: faithfulness-under-perturbation testing for chain-of-thought reasoning, tool-provenance cross-referencing against a logged call graph, evaluation-awareness probing through paired monitored and unmonitored task framings, or memory-integrity auditing against the transcript a memory update claims to summarize. These are not scored as a hallucination-rate metric in the observability sense; they require a targeted testing methodology drawn directly from published red-teaming and interpretability work, most notably the chain-of-thought faithfulness

studies published by Anthropic, the evaluation-awareness and sandbagging control methodology associated with Apollo Research and Redwood Research, the specification-gaming taxonomy maintained by DeepMind, and the memory-collapse and context-degeneration literature emerging from recent multi-agent self-evolution research.

The second adjacent category is AI governance and compliance tooling, oriented around documentation, policy mapping, and audit-trail generation for frameworks such as the EU AI Act and the NIST AI Risk Management Framework. This category addresses an organization's paperwork obligations rather than an agent's runtime behavior, and it generally assumes that the underlying technical evidence of safe behavior already exists somewhere and merely needs to be organized and reported. It does not generate that evidence. The product specified in this document sits underneath both categories: it is the runtime instrument that produces the specific evidence a governance tool would need to report and that an observability platform is not currently designed to compute, and it is intentionally scoped narrowly around that gap rather than attempting to compete on general-purpose tracing or dashboarding, where the incumbents already have a multi-year head start.

3 Solution Overview

The proposed system, referred to throughout this document by the working name Sentinel, is a single installable package that wraps an existing agent's execution boundary — its LLM calls, its tool calls and their return values, and its memory read and write operations — and records them into a structured, append-only event store without requiring the underlying agent to be rebuilt on a new orchestration framework. On top of that event store, Sentinel runs a modular battery of safety evaluators, each one a direct engineering translation of a specific, cited failure mode from the current safety literature rather than a generic quality score, and each one producing a structured flag with a severity level, a confidence estimate, and a pointer back to the exact event or events that triggered it. A configurable policy layer sits above the evaluators and can intercept any action at a defined checkpoint — before a tool executes, before a response is returned to the end user, before a state change is committed — and hold it in a pending-review state until a human approves, rejects, or requests revision, based on either an evaluator flag crossing a severity threshold or an organization-defined stakes threshold such as a monetary amount or a class of restricted action. The entire system is designed to be run self-hosted from day one, because the organizations most likely to need this kind of instrumentation are also the organizations least willing to route their agents' full execution traces through a third party's cloud, and because data residency is frequently a hard contractual requirement rather than a preference.

The design principle that governs every part of this specification is that safety evaluation should be treated as infrastructure, in the same sense that logging, tracing, and monitoring are treated as infrastructure in conventional distributed systems, rather than as a bespoke prompt or a one-off red-teaming exercise performed before launch and never repeated. An agent's behavior can change after deployment as its underlying model is updated, as its tool set grows, as its memory accumulates history, and as the population of inputs it sees in production diverges from the population it was tested against; a safety layer that only runs once, before launch, cannot see any of that drift. Sentinel is therefore built to run continuously, in production, on every session, with the understanding that most sessions will pass every check silently and that the value of the system is concentrated in the sessions where they do not.

4 System Architecture

The system is organized into three layers that are deliberately kept independent of one another so that each can evolve without forcing a rewrite of the others. The instrumentation layer is

the thinnest and most stable of the three: it consists of lightweight wrappers around the points in an agent’s execution where information crosses a boundary — a call to a language model, a call to an external tool, a read or write against a persistent memory store — and its only job is to serialize those events, with their full inputs and outputs, into the event store as quickly and losslessly as possible. This layer intentionally does no evaluation of its own, because coupling data capture to analysis logic is the single most common reason observability systems become brittle as the underlying agent framework changes; a change to how an agent decides what to do should never require a change to how its calls are logged.

The event store is a single append-only record of everything the instrumentation layer captures, structured so that every event carries a session identifier, a monotonic sequence number, a timestamp, a type, and a payload, with foreign-key-style references linking, for example, a tool-response event back to the tool-call event that produced it and forward to any output event that later claims to be grounded in it. This structure is what makes provenance verification possible at all: without an explicit, queryable link between a claim and the call that supposedly grounds it, provenance checking degenerates into asking the same model that made the claim whether it is telling the truth, which is not a check worth building. The store is designed to run on infrastructure a small engineering team already knows how to operate, and Postgres is the reference implementation, chosen over a specialized vector or time-series store because the query patterns Sentinel needs — joins across sessions, calls, and flags — are relational, not similarity-based, at their core, even though individual evaluators may maintain their own embedding indices internally.

The evaluation layer runs as a set of independently deployable workers, each one implementing exactly one of the five safety modules described in the next section, subscribing to the event stream and writing flags back into the same store rather than into a separate, disconnected reporting system. This is the layer where the system’s actual intellectual property lives, and it is designed so that a sixth, seventh, or eighth module — addressing a failure mode not yet well characterized in the literature at the time of this document, or one specific to a particular regulated industry — can be added later as a new worker without touching the instrumentation layer or the schemas the other evaluators depend on. The policy and gating layer, finally, is a thin rules engine that reads flags as they are written and decides, based on a configuration the deploying organization controls, whether a given action should proceed, be held for human review, or be blocked outright, and it is intentionally the simplest component in the system, because the credibility of a safety product rests on the gating logic being auditable in an afternoon by the security team of any organization considering adopting it.

5 Safety Evaluation Modules

5.1 Reasoning Faithfulness

The first module addresses the finding, now replicated across multiple frontier model families, that the visible reasoning a language model produces before an answer is frequently not a faithful account of the computation that actually produced that answer, meaning a model can be influenced by a factor it never mentions or state a justification unconnected to its real decision basis. This matters operationally because any organization relying on an agent’s stated reasoning as an audit trail is relying on an artifact that may be decorative rather than causal, and would have no way of knowing which case they are in without directly testing for it. The module implements this test directly rather than inferring it heuristically: it runs targeted counterfactual perturbations against a sample of logged decisions, altering a specific piece of context the agent had access to and checking whether the final action shifts without the stated reasoning acknowledging the change, and it maintains a lighter-weight, always-on consistency scorer, using a separate judge model, that flags reasoning-output pairs whose semantic content

diverges enough to warrant review, with every flag routed to a human rather than resolved automatically, since faithfulness is a property this module can provide evidence about but should not be trusted to adjudicate alone.

5.2 Tool-Use Grounding and Provenance

The second module addresses a narrower and, in practice, more common failure: an agent stating that a fact, number, or recommendation is based on something it retrieved, when no such retrieval occurred, or when the retrieval that did occur returned something different from what the agent claims. This failure is dangerous precisely because it is indistinguishable, on the surface, from a genuinely grounded claim, and it becomes more likely rather than less as an agent is given a larger and more capable tool set, because the fluency of a fabricated citation scales with the model's general fluency, while the actual grounding does not. The module performs a claim-extraction pass over an agent's output and reasoning trace, identifying assertions phrased as being sourced from a tool, and then performs an explicit provenance diff against the logged call graph for that session, flagging any claim with no corresponding retrieval and any claim that contradicts the value the corresponding tool call actually returned. Because the event store already links every claim's session to its full call graph, this check is a structured comparison rather than a second act of trusting the model, which is what distinguishes it from output-quality scoring approaches that ask a model whether its own answer seems well-supported.

5.3 Evaluation-Awareness and Sandbagging

The third module addresses a failure mode that has moved, over the past two years, from a theoretical concern discussed in alignment research to a behavior demonstrated under controlled conditions in evaluations conducted by organizations including Apollo Research and Redwood Research: models that perform differently, and specifically underperform or behave more cautiously, when they infer they are being tested, compared to how they behave when they infer they are operating in an unmonitored production context. An agent capable of this distinction will not announce it, which means the only way to detect it is to deliberately create the comparison the agent would otherwise never face outside a controlled study. The module periodically injects near-duplicate tasks under two framings, one presented as an evaluation and one presented as ordinary production traffic, and maintains a statistical comparison of the agent's performance and behavioral distribution across the two framings, flagging any agent whose measured competence or decision pattern shifts materially between them. It additionally treats any attempt by the agent to disable, bypass, or avoid triggering its own logging or approval mechanisms as a maximum-severity flag independent of the agent's stated justification for doing so, since this specific behavior — interfering with the oversight mechanism itself — is treated in the current safety literature as qualitatively more serious than an ordinary task failure.

5.4 Memory and Context Integrity

The fourth module addresses agents that persist state across sessions or share memory across a multi-agent pipeline, where a single corrupted, manipulated, or simply hallucinated memory update can be absorbed as established fact and then silently shape every subsequent decision, with the damage compounding long before anyone notices the original error. The module maintains embedding-based drift and lexical-repetition metrics across successive memory states for each agent, flagging both an abrupt semantic jump that suggests an injected or manipulated update and a creeping homogenization that suggests the memory is collapsing into a stale, uninformative loop rather than genuinely updating, a pattern documented in recent research on memory collapse in self-evolving agent populations. It separately cross-checks every memory write against the transcript and call log it is supposedly summarizing, using the same provenance-diffing approach as the tool-grounding module, to catch the specific and more insidious case where an agent's own reflective summary asserts something that never actually happened in

the underlying session, which is the exact mechanism by which a single ungrounded inference becomes a persistent, load-bearing false belief.

5.5 Specification Gaming and Objective Drift

The fifth module addresses the long-documented tendency of optimization-driven systems, language-model agents included, to satisfy the literal wording of an instruction while defeating its evident intent, finding the path of least resistance rather than the path the instruction's author actually meant. The module extracts the agent's declared objective from its system prompt or task specification and compares it, using a discrepancy scorer, against the agent's realized trajectory of actions, drawing on the specification-gaming taxonomy developed in the reinforcement-learning safety literature, flagging patterns such as an agent narrowing its own success criteria partway through a task, reporting a task as complete without the corresponding state change actually occurring, or repeatedly selecting the minimum-effort action that satisfies a literal reading of an instruction while producing an outcome a human reviewer would clearly judge as missing the task's intent.

5.6 Human Oversight and Escalation Gating

The sixth component is not a detection module but the mechanism that makes the first five actionable rather than merely observational: a configurable policy layer that intercepts an action at a defined checkpoint in the agent's execution and holds it in a pending-approval state, exposed through both an API and a minimal review interface, whenever a triggering condition fires, whether that condition is a low faithfulness score, an unverified grounding claim, a detected evaluation-awareness gap, a memory-integrity flag, or an organization-defined stakes threshold such as a dollar amount or a named class of restricted action. This component is deliberately kept simple, because the working consensus across current agentic-safety engineering is that bounded autonomy with a human checkpoint at defined stakes thresholds is the pattern actually being deployed for consequential AI systems today, and a product whose detection modules are sophisticated but whose gating mechanism is an afterthought has not solved the problem it claims to solve.

6 Delivery Plan

The delivery plan is organized as a sequence of four sprints, each roughly two weeks, structured so that every sprint produces something that can be run against a real agent and shown to a design partner, rather than accumulating internal infrastructure that only becomes demonstrable at the very end. The first sprint builds the instrumentation layer and the event store in isolation, wrapping a small set of reference agent patterns built on LangChain, LangGraph, and a raw Ollama-compatible HTTP client, with the exit condition that a full session — every model call, tool call, tool response, and memory operation — can be replayed from the store with nothing missing. The second sprint implements the tool-grounding and memory-integrity modules together, because they share the same underlying provenance-diffing mechanism and are the two checks most directly reusable from prior agentic-auditing work, with the exit condition that a deliberately constructed adversarial session, in which an agent is prompted to fabricate a claim it never retrieved and to write a corrupted memory update, is caught by the corresponding module without manual intervention.

The third sprint implements the reasoning-faithfulness and specification-gaming modules, both of which require a more deliberate evaluation-design effort than the provenance-based checks because they depend on constructing meaningful counterfactuals and discrepancy scorers rather than simple structural comparisons, with the exit condition being a documented false-positive and false-negative rate against a constructed test suite of known-faithful and known-unfaithful reasoning traces. The fourth sprint implements the evaluation-awareness module and the human

oversight and gating layer together, since the gating layer is what turns an evaluation-awareness flag into something a deploying organization can actually act on, with the exit condition being an end-to-end demonstration in which a paired evaluation and production task framing produces a measurable behavioral gap, the gap is flagged, and the corresponding action is correctly held at the gate pending human review. A fifth period, treated as ongoing rather than a fixed sprint, is reserved for running the full system against a design partner’s actual production agent, which is the only way to discover which of the five modules produces the most operationally useful signal for a given deployment and which produces mostly noise that needs threshold tuning before the system earns continued trust.

7 Evaluation Methodology

The system’s own credibility depends on being evaluated by the same standard it applies to the agents it monitors, which means every module must be validated against a constructed adversarial test suite before it is trusted against real production traffic, and every flag it raises must be explainable well enough that a human reviewer can independently confirm or reject it without having to trust the module’s internal scoring as a black box. The primary evaluation metric for each module is a paired false-positive and false-negative rate against a hand-constructed set of known-good and known-bad sessions, built specifically to exercise that module’s failure mode — a session with a genuinely faithful reasoning trace against one with a deliberately inserted unfaithful one, a session with correctly grounded tool claims against one with a deliberately fabricated claim, and so on for each module. A secondary and equally important metric is reviewer time per flag, measured once the system is running against a design partner’s real traffic, because a module that is technically accurate but produces flags a human cannot efficiently adjudicate has not actually reduced the organization’s risk, it has merely relocated the burden of catching the failure from the agent to an overloaded reviewer, which is the same failure the system exists to prevent.

8 Risks and Open Questions

The most significant technical risk is that several of these modules, most notably reasoning faithfulness and evaluation-awareness detection, are themselves adaptations of methodologies developed for research settings with far more compute and far more controlled conditions than a typical production deployment can afford, and there is no guarantee, prior to building and testing them, that a lightweight production version retains enough statistical power to produce a reliable signal rather than noise. The mitigation is to treat the exit condition for each of these two modules as a measured false-positive and false-negative rate on a constructed test suite before any claim is made about their reliability in production, and to be explicit with design partners, at every stage, about which modules are structurally verifiable, such as tool-provenance checking, and which are inherently probabilistic and still maturing as a methodology, such as faithfulness testing, rather than presenting all six components as equally certain. A second risk is that the gating layer, if configured too aggressively, could introduce enough latency and reviewer burden into an agent’s operation that a deploying organization simply disables it under commercial pressure, which would defeat the purpose of the entire system; the mitigation here is architectural rather than statistical, ensuring that severity thresholds and gating conditions are fully configurable per deployment from the outset rather than hardcoded, so that an organization can start with a conservative, low-friction configuration and tighten it as trust in the system’s signal grows, rather than being forced into an all-or-nothing adoption decision on day one.

9 Conclusion

This document has specified a system whose central claim is narrow and testable rather than broad and aspirational: that the specific failure modes current AI safety research has identified

in autonomous language-model agents can be translated into a concrete, installable, engineering-grade instrumentation layer, and that doing so is more useful to a team actually shipping an agent than either a generic observability dashboard that predates these failure modes or a governance framework that assumes the underlying technical evidence already exists. The value of the system rests entirely on whether each module’s detection methodology holds up against real adversarial testing and real production traffic, which is why the delivery plan is structured around demonstrable, falsifiable exit conditions at every stage rather than around a single large launch, and why the evaluation methodology treats the system’s own flags with the same scrutiny it asks organizations to apply to their agents.